OpenArm v1 Classic CAN 1Mbps & ExoArm 착용형 제어 명령어

장소 이동 후 전원/CAN 재설정, zero calibration, ExoArm ROS2-HMI-OpenArm bridge 실행 순서

핵심 모드	OpenArm v1 기준 Classic CAN 1Mbps 사용. 이 절차에서는 dbitrate 5000000 fd on을 넣지 않습니다.
캘리브레이션	기본 calibration 스크립트가 CAN-FD용이면 oa.OpenArm(args.canport, True)를 False로 바꾼 classic script를 사용합니다.
HMI 주소	HMI 포워딩 WebSocket IP는 ws://127.0.0.1:19091 입니다.
비상 정지	언제든 아래 disable 두 줄을 즉시 실행할 수 있게 별도 터미널에 준비합니다.

번호	단계	내용
0	초기 disable	전원/CAN 재연결 직후 can0/can1 모터 비활성화
1	CAN 설정	can0/can1을 Classic CAN 1Mbps로 down-type-up
2	classic calibration script	zero_calib_classic.py 생성 및 OpenArm(..., False) 적용
3	오른팔 calibration	can0 / right_arm
4	왼팔 calibration	can1 / left_arm
5	sampling check	can0/can1 motor sampling check
6	ExoArm 실행	ROS2 launch -> HMI -> OpenArm bridge

맨 아래 Appendix에는 현장에서 그대로 따라 치기 쉬운 형태로 전체 명령어를 다시 모았습니다.

주의: 실제 로봇 구동 전 물리 E-stop을 손 닿는 곳에 두고, CAN 포트 매핑과 arm-side를 현장에서 재확인하세요.

A. 장소 이동 후 전원/CAN 재설정 및 Classic CAN 캘리브레이션

로봇 위치를 옮겼거나 전원을 다시 넣은 경우, 먼저 can0/can1을 비활성화한 뒤 Classic CAN 1Mbps로 다시 잡습니다.

0) 비상 정지 / 초기 disable

```
openarm-can-cli -i can0 disable
openarm-can-cli -i can1 disable
```

1) CAN0/CAN1 Classic CAN 1Mbps 설정

```
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 restart-ms 100
sudo ip link set can0 up

sudo ip link set can1 down
sudo ip link set can1 type can bitrate 1000000 restart-ms 100
sudo ip link set can1 up
```

2) classic calibration 스크립트가 없으면 생성

```
cp /usr/bin/openarm-can-zero-position-calibration ~/zero_calib_classic.py
sed -i 's/oa.OpenArm(args.canport, True)/oa.OpenArm(args.canport, False)/' ~/zero_calib_classic.py
chmod +x ~/zero_calib_classic.py
```

확인 포인트

- sed 명령이 True를 False로 바꿔 Classic CAN 경로를 사용하게 합니다.
- 이 절차는 OpenArm v1 / Classic CAN 1Mbps 기준입니다.
- 스크립트 실행 전 로봇 팔 자세, 그리퍼 상태, 물리 E-stop 위치를 확인합니다.

B. 오른팔/왼팔 zero calibration 실행 및 정상 확인

3) 오른팔 can0 캘리브레이션

```
~/zero_calib_classic.py --canport can0 --arm-side right_arm
```

4) 왼팔 can1 캘리브레이션

```
~/zero_calib_classic.py --canport can1 --arm-side left_arm
```

정상 문구 – 원문 출력 그대로

```
wrote zero positon to arm
[INFO] Motors disabled, exiting safely.
```

주의: 정상 문구의 'positon' 오탈자는 실제 출력 문구를 그대로 적은 것입니다.

확인 – motor sampling check

```
cd ~/openarm_can/build

./openarm-can-motor-sampling-check --all 100 can0
./openarm-can-motor-sampling-check --all 100 can1
```

비상 정지 – 언제든 실행

```
openarm-can-cli -i can0 disable
openarm-can-cli -i can1 disable
```

현장 체크

- can0 = 오른팔, can1 = 왼팔 기준으로 정리했습니다.
- CAN 어댑터 순서가 바뀌면 can 번호도 바뀔 수 있으므로 실행 전 확인합니다.
- sampling check가 끝나면 다음 단계인 ExoArm ROS2/HMI/bridge 실행으로 넘어갑니다.

C. ExoArm 착용형 제어 실행 순서

터미널 3개를 각각 열고 아래 순서대로 실행합니다. HMI에서는 포워딩 주소를 ws://127.0.0.1:19091로 맞춥니다.

터미널 1 – ExoArm ROS2

```
cd ~/foxtech_exoarm/qnbot-exo-std-demo
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch qnbot_teleoperator websocket_teleoperator.launch.py
```

터미널 2 – HMI

```
cd ~/foxtech_exoarm/software-tool-v1.2.3/linux
./"Qnbot HMI Control-1.2.3.AppImage"

# HMI 포워딩 IP / WebSocket 주소
ws://127.0.0.1:19091
```

HMI 포워딩 IP

ws://127.0.0.1:19091

HMI Control에서 WebSocket/forwarding 주소로 이 값을 넣습니다.

터미널 3 – OpenArm CAN 세팅 + 브리지

```
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 restart-ms 100
sudo ip link set can0 up

sudo ip link set can1 down
sudo ip link set can1 type can bitrate 1000000 restart-ms 100
sudo ip link set can1 up

python3 ~/openarm_teleop/exo_openarm_bridge/exo_to_openarm.py
```

실행 중단/재시작 원칙

- 로봇이 예상과 다르게 움직이면 즉시 can0/can1 disable 명령을 실행합니다.
- bridge 재실행 전에는 기존 ROS2 launch, HMI, bridge 프로세스가 중복 실행 중인지 확인합니다.
- 장소를 다시 옮겼거나 전원을 껐다 켰다면 CAN 설정과 sampling check부터 다시 수행합니다.

Appendix 1. 현장 복붙용 – 캘리브레이션

아래 블록은 장소 이동 후 전원/CAN 재설정부터 zero calibration, 확인, 비상 정지까지의 명령어입니다.

```
# 0) 비상 정지/초기 disable
openarm-can-cli -i can0 disable
openarm-can-cli -i can1 disable
```

```
# 1) CAN0/CAN1 Classic CAN 1Mbps 설정
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 restart-ms 100
sudo ip link set can0 up

sudo ip link set can1 down
sudo ip link set can1 type can bitrate 1000000 restart-ms 100
sudo ip link set can1 up
```

```
# 2) classic calibration 스크립트 없으면 생성
cp /usr/bin/openarm-can-zero-position-calibration ~/zero_calib_classic.py
sed -i 's/oa.OpenArm(args.canport, True)/oa.OpenArm(args.canport, False)/' ~/zero_calib_classic.py
chmod +x ~/zero_calib_classic.py
```

```
# 3) 오른팔 can0 캘리브레이션
~/zero_calib_classic.py --canport can0 --arm-side right_arm

# 4) 왼팔 can1 캘리브레이션
~/zero_calib_classic.py --canport can1 --arm-side left_arm
```

```
# 정상 문구
wrote zero positon to arm
[INFO] Motors disabled, exiting safely.
```

주의: 실제 로봇 구동 전 물리 E-stop을 손 닿는 곳에 두고, CAN 포트 매핑과 arm-side를 현장에서 재확인하세요.

Appendix 2. 현장 복붙용 – 확인 및 ExoArm 실행

sampling check 후, 터미널 1-2-3 순서로 ExoArm ROS2, HMI, OpenArm bridge를 실행합니다.

```
# 확인
cd ~/openarm_can/build

./openarm-can-motor-sampling-check --all 100 can0
./openarm-can-motor-sampling-check --all 100 can1

# 비상 정지
openarm-can-cli -i can0 disable
openarm-can-cli -i can1 disable
```

```
# 터미널 1 — ExoArm ROS2
cd ~/foxtech_exoarm/qnbot-exo-std-demo
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch qnbot_teleoperator websocket_teleoperator.launch.py
```

```
# 터미널 2 — HMI
cd ~/foxtech_exoarm/software-tool-v1.2.3/linux
./"Qnbot HMI Control-1.2.3.AppImage"

# HMI 포워딩 IP / WebSocket 주소
ws://127.0.0.1:19091
```

```
# 터미널 3 — OpenArm CAN 세팅 + 브리지
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 restart-ms 100
sudo ip link set can0 up

sudo ip link set can1 down
sudo ip link set can1 type can bitrate 1000000 restart-ms 100
sudo ip link set can1 up

python3 ~/openarm_teleop/exo_openarm_bridge/exo_to_openarm.py
```

주의: 실제 로봇 구동 전 물리 E-stop을 손 닿는 곳에 두고, CAN 포트 매핑과 arm-side를 현장에서 재확인하세요.